

Função wait_for_ack():

```
def wait_for_ack(expected_vest_id, timeout):
    # Cronómetro para esperar pelo ACK
    start_time = time.time()
    while (time.time() - start_time) < timeout:
        packet = hardware_receive(timeout=0.1, max_bytes=2)
    # ACKs têm sempre 2 bytes
    if packet is not None and len(packet) == 2:
        decrypted = decrypt_aes_ctr(packet)
        received_vest_id = decrypted[0] & 0x1F
        opcode = (decrypted[1] >> 4) & 0x0F
        funct = decrypted[1] & 0x0F
        # Se for o colete certo e o opcode for 0b1001,
        deu certo
        if received_vest_id == expected_vest_id and opcode == 0b1001 and funct == 0b0010:
            return True
    return False
```

```
def wait_for_ack(expected_vest_id, timeout):
    start_time = time.time()
    while (time.time() - start_time) < timeout:
```

- **expected_vest_id** e **timeout**: A função está à espera por exemplo do Colete 5 e quanto tempo máximo pode esperar (ex: 2.5 segundos).
- **start_time = time.time()**: Regista a hora exata em que o relógio começou a contar.
- **while (...) < timeout**: Enquanto a diferença entre a hora atual e a hora de início for menor que o tempo limite, continua a tentar ouvir.

```
packet = hardware_receive(timeout=0.1, max_bytes=2) # ACKs
têm sempre 2 bytes
    if packet is not None and len(packet) == 2:
        decrypted = decrypt_aes_ctr(packet)
        received_vest_id = decrypted[0] & 0x1F
        opcode = (decrypted[1] >> 4) & 0x0F
        funct = decrypted[1] & 0x0F
```

- **hardware_receive(timeout=0.1, max_bytes=2)**: Em vez de ficar parado durante os 2.5 segundos inteiros, ele ouve em blocos rápidos de 0.1 segundos. Isto evita que o programa encrave. E só aceita mensagens que tenham exatamente um tamanho máximo de 2 bytes.
- **if packet is not None and len(packet) == 2**: Isto é uma verificação de segurança. Só avança se a mensagem não for vazia e se tiver um tamanho exato de **2 bytes** (ACK).
- **decrypted = ...**: Descripta para podermos ler.
- **received_vest_id = decrypted[0] & 0x1F**: O **byte 0** tem a identificação. Ao fazer **& 0x1F**, ignoramos os primeiros 3 bits que são o NetID e ficamos apenas com os últimos 5 bits, que guardam o número do colete que acabou de falar.
- **opcode = (decrypted[1] >> 4) & 0x0F**: O **byte 1** tem o comando que o colete está a responder. Pegamos nesse byte 1 e empurrámos 4 casas para a direita. Com a máscara **& 0x1F**, lemos esse valor (Opcode)